ECAG-Bench: A Benchmark for Constructive Algebraic Geometry Reasoning in Large Language Models

Waerden* Claude Code†

Abstract

We introduce ECAG-Bench, a benchmark for evaluating the ability of large language models to perform *constructive reasoning* in algebraic geometry. Unlike existing mathematical benchmarks that focus on theorem proving or competition problems, ECAG-Bench requires models to construct concrete mathematical objects—schemes, sheaves, morphisms, and cohomology classes—satisfying specified properties. The benchmark comprises 332 problems derived from a curated collection of examples and counterexamples in algebraic geometry, organized across four chapters and three difficulty levels. A distinctive feature is the dual emphasis on *examples* (243 problems asking models to build objects with prescribed properties) and *counterexamples* (66 problems requiring models to construct objects that violate plausible-sounding statements), testing whether models understand not only where theorems apply but also where they fail. Our four-layer verification pipeline—parsing, computer algebra system (CAS) verification, LLM-assisted judgment, and human expert sampling—validates properties of constructed objects rather than checking answer equivalence. We report results from a pilot evaluation of Claude Sonnet 4.6 on a stratified 100-problem subset, finding an overall accuracy of 78% with a pronounced gap between example construction (80%) and counterexample construction (65%), and a steep difficulty gradient from Level 1 (95%) to Level 3 (27%), suggesting that constructive reasoning at the frontier of algebraic geometry remains substantially beyond current model capabilities. We release the full dataset, evaluation code, and an accompanying open-source textbook at `https://github.com/Waerden001/Examples_and_Counterexamples_in_Elementary_Algebraic_Geometry`.

Keywords

Large language models, mathematical reasoning, algebraic geometry, benchmark, constructive mathematics

1 Introduction

Mathematical reasoning has emerged as a central testbed for evaluating the capabilities of large language models (LLMs). Existing benchmarks primarily assess two modes of mathematical reasoning: *competition-style problem solving* [Hendrycks et al., 2021, Zhong et al., 2023] and *formal theorem proving* [Tsoukalas et al., 2024, Zheng et al., 2022]. Both modes share a common structure: given a precisely stated problem, produce a proof or a unique numerical answer.

We identify a third mode—*constructive reasoning*—that is pervasive in mathematical practice yet largely absent from current benchmarks. Constructive reasoning asks: given a set of desired properties, can you *build* a concrete mathematical object that satisfies them? For example:

- Construct a smooth projective curve of genus 2 whose Jacobian has complex multiplication.

*Independent Researcher. Correspondence: `42280958+Waerden001@users.noreply.github.com`

†Anthropic. This work was conducted using Claude Code as a research collaborator.

- Give an example of a flat morphism that is not smooth.
- Find a coherent sheaf on $\mathbf{P}^2$ with prescribed Hilbert polynomial.
- Exhibit a reduced scheme that is not geometrically reduced.
- Give a counterexample to the statement: every coherent sheaf on a smooth variety is locally free.

This mode of reasoning is fundamentally different from verification. In verification, one checks whether a given object has a property; in construction, one must *synthesize* an object from scratch. The answer space is typically infinite and non-unique: many different objects may satisfy the given constraints, and the "difficulty" lies not in finding *the* answer but in finding *any* valid answer.

The role of counterexamples. Constructive reasoning encompasses not only building objects that *satisfy* a theorem's hypotheses, but also constructing objects that *violate* plausible-sounding claims—that is, producing counterexamples. As Lakatos [Lakatos, 1976] argued in *Proofs and Refutations*, the interplay between conjectures and their refutations is the engine of mathematical progress: a theorem is truly understood only when one also understands *where it fails*. The history of algebraic geometry is rich with counterexamples that reshaped the field: Nagata's counterexample to Hilbert's fourteenth problem [Nagata, 1959], Hironaka's example of a non-projective complete variety [Hironaka, 1964], and the discovery of non-smoothable singularities all advanced the subject by delineating the precise boundaries of general theorems. Constructing a counterexample requires a deeper understanding than constructing a positive example: one must identify exactly which hypothesis of a theorem is violated and build an object that satisfies all other conditions while failing that specific one. This "boundary awareness" is precisely what we aim to test.

Algebraic geometry provides an ideal domain for testing constructive reasoning. The field is characterized by a rich interplay between abstract theory and concrete computation: theorems about schemes, sheaves, and cohomology become meaningful only when instantiated on specific geometric objects. Standard references [Hartshorne, 1977, Vakil, 2017, Stacks project authors, 2024, Eisenbud, 1995, Griffiths and Harris, 1978] develop the theory through carefully chosen examples that illustrate both the power and the limitations of general results.

We introduce ECAG-Bench (**E**xamples and **C**ounterexamples in **A**lgebraic **G**eometry **Bench**mark), a dataset of 332 constructive problems in algebraic geometry. Each problem asks the model to construct a specific mathematical object—a scheme, a morphism, a sheaf, a cohomology class, or a numerical invariant—satisfying stated properties. The problems are drawn from *Examples and Counterexamples in Elementary Algebraic Geometry*, a curated collection developed over several years as supplementary material for learners entering the subject. A distinctive feature of ECAG-Bench is its dual structure: 243 problems ask for *examples* (construct an object with prescribed properties) while 66 problems ask for *counterexamples* (construct an object violating a plausible statement), with the remaining 23 problems involving computation or proof. This examples-versus-counterexamples dichotomy reflects the natural structure of algebraic geometry, where understanding a concept requires seeing both what it includes and what it excludes.

Project ecosystem. ECAG-Bench is part of an integrated open-source project comprising: (i) a LaTeX textbook of worked examples and counterexamples in algebraic geometry; (ii) a GitHub Pages website providing searchable, MathJax-rendered content; (iii) the benchmark dataset in JSONL format with full metadata and verification strategies; and (iv) evaluation tooling for running and scoring LLM responses. All components are released under the CC-BY-4.0 license.

Our key contributions are:

1. **A new benchmark** of 332 constructive algebraic geometry problems at three difficulty levels, covering four major areas of the subject, with an explicit dual structure of examples

Table 1: Comparison of mathematical reasoning benchmarks. "Constructive?" indicates whether problems require constructing mathematical objects (as opposed to producing proofs or unique answers). "Counterex." indicates whether counterexample construction is explicitly tested.

Benchmark	**Size**	**Domain**	**Verification**	**Construct.?**	**Counte**
MATH [Hendrycks et al., 2021]	12,500	Competition math	Exact match	×	×
AGIEval [Zhong et al., 2023]	8,062	Standardized tests	Multiple choice	×	×
miniF2F [Zheng et al., 2022]	488	Olympiad math	Proof assistant	×	×
PutnamBench [Tsoukalas et al., 2024]	1,697	Putnam competition	Lean/Isabelle/Coq	×	×
FrontierMath [Glazer et al., 2024]	300+	Research math	Custom scripts	Partial	×
GHOSTS [Zheng et al., 2023]	700+	Graduate science	Rubric	×	×
AMBER [Liu et al., 2025]	500+	Lean 4 math	Lean 4	✓	×
MathArena [Balunović et al., 2025]	Varies	Recent competitions	Dual grading	×	×
ECAG-Bench (ours)	332	Algebraic geometry	CAS + rubric	✓	✓

(243) and counterexamples (66).

2. **A four-layer verification pipeline** that validates properties of constructed objects rather than checking answer equivalence, using computer algebra systems (CAS) as the primary verification mechanism.
3. **Pilot evaluation results** from Claude Sonnet 4.6 on a stratified 100-problem subset, establishing initial baselines and revealing a significant performance gap between example construction (80%) and counterexample construction (65%), with a steep difficulty gradient from Level 1 (95%) to Level 3 (27%).
4. **A taxonomy of six failure modes** specific to constructive mathematical reasoning, including fabricated objects (F1), property confusion (F2), computational errors (F3), trivial answers (F4), hallucinated theorems (F5), and incomplete verification (F6).
5. **An open-source collection** of 332 fully worked problems with an accompanying textbook and website, designed to support both LLM evaluation and algebraic geometry education.

Paper structure. Section 2 reviews related work in mathematical benchmarks and positions ECAG-Bench relative to existing evaluations. Section 3 describes the dataset, its sources, and the examples/counterexamples duality. Section 4 presents the benchmark design including the verification pipeline. Section 5 reports our pilot evaluation results, and Section 6 provides detailed analysis of failure modes and performance patterns. Section 7 concludes with limitations and future directions.

2 Related Work

2.1 Mathematical Reasoning Benchmarks

Table 1 provides a systematic comparison of ECAG-Bench with existing mathematical reasoning benchmarks.

Competition and undergraduate mathematics. The MATH dataset [Hendrycks et al., 2021] evaluates competition-level mathematics across seven subjects, with problems requiring numerical or symbolic answers. AGIEval [Zhong et al., 2023] includes math problems from standardized tests. These benchmarks test *problem-solving* ability but focus on high-school and undergraduate competition mathematics, where problems have unique answers and standard solution techniques. Frieder et al. [2023] systematically evaluated ChatGPT's mathematical capabilities, finding significant gaps in reasoning even on undergraduate-level problems.

Research-level mathematics. FrontierMath [Glazer et al., 2024] is the closest benchmark in spirit, comprising research-level mathematics problems with custom verification scripts. However, FrontierMath spans all areas of mathematics and does not specifically target constructive reasoning; many problems still ask for unique numerical answers. GHOSTS [Zheng et al., 2023] evaluates graduate-level science tasks using rubric-based scoring, but covers mathematics only as one component of a broader scientific evaluation. Minerva [Lewkowycz et al., 2022] demonstrated that language models pretrained on scientific text could achieve strong performance on quantitative reasoning, spurring interest in more challenging mathematical benchmarks.

Formal theorem proving. PutnamBench [Tsoukalas et al., 2024] formalizes Putnam competition problems in Lean, Isabelle, and Coq, testing whether LLMs can produce machine-verifiable proofs. AMBER [Liu et al., 2025] combines construction and verification in Lean 4, requiring models to both state and prove lemmas. miniF2F [Zheng et al., 2022] provides a cross-system benchmark for Olympiad-level formal mathematics. NaturalProver [Welleck et al., 2022] and ProofNet [Azerbayev et al., 2023] explore the interface between informal and formal mathematical reasoning. These benchmarks operate in the formal verification paradigm, where correctness is checked by a proof assistant. ECAG-Bench differs by operating in the *informal* mathematical setting, using CAS verification of object properties rather than formal proof checking.

Interactive and multi-turn evaluation. Collins et al. [2024] propose evaluating mathematical capabilities through multi-turn interactions. MathArena [Balunović et al., 2025] uses recent competition problems with dual grading (automatic and human) to minimize contamination. ECAG-Bench currently uses single-turn evaluation but is designed to support multi-turn interaction in future work.

2.2 Domain-Specific Benchmarks

While general mathematical benchmarks span many areas, there is growing recognition that domain-specific evaluation is essential for measuring deep mathematical competence. AlphaGeometry [Trinh et al., 2024] demonstrated that combining neural language models with symbolic engines can solve Olympiad-level geometry problems, but geometry in the Euclidean sense is a substantially different domain from algebraic geometry over arbitrary fields and rings. To our knowledge, ECAG-Bench is the *first* benchmark specifically targeting algebraic geometry—a field that sits at the intersection of algebra, geometry, and topology, requiring fluency with abstract categorical constructions alongside concrete polynomial computations.

The domain specificity of ECAG-Bench is both a limitation and a strength. On one hand, results do not directly generalize to other mathematical domains. On the other hand, depth within a single domain allows us to calibrate difficulty levels against the actual pedagogical structure of algebraic geometry and to design verification strategies that exploit the domain's computational tools (SageMath [The Sage Developers, 2024], Macaulay2 [Grayson and Stillman, 2024], Singular [Decker et al., 2024]).

2.3 Constructive Reasoning and Counterexamples

The distinction between constructive and verificational reasoning is well-established in mathematics [Hartshorne, 1977] but has received limited attention in the LLM evaluation literature. Lakatos [Lakatos, 1976] placed counterexamples at the center of mathematical methodology, arguing that mathematical concepts are refined through a dialectical process of conjecture, attempted proof, and refutation by counterexample. In algebraic geometry, this process is especially pronounced: many fundamental results (e.g., Zariski's Main Theorem, cohomological criteria for ampleness) are accompanied by canonical counterexamples that delineate their precise scope.

ECAG-Bench explicitly tests both directions of this dialectic: 243 problems ask for examples ("construct an object satisfying...") and 66 ask for counterexamples ("construct an object violating..."). This dual structure is, to our knowledge, unique among mathematical benchmarks. As we show in Section 5, the gap between model performance on examples versus counterexamples is substantial and informative.

2.4 Contamination Considerations

A persistent concern in LLM evaluation is *data contamination*: test problems that appear in the training corpus. ECAG-Bench has several properties that mitigate this risk:

1. The source material is a custom-written collection of examples, not drawn from standard problem sets, competition archives, or widely-distributed textbooks.
2. Problems are embedded as structured annotations within the source material and extracted programmatically; the specific problem formulations do not exist as standalone documents on the web.
3. The content enhancement process that produced the final problem statements was performed using the Anthropic API, generating unique formulations that are unlikely to match any training data verbatim.
4. The source repository has been public for a limited period, reducing the probability that its content entered pre-training corpora for currently evaluated models.

While no benchmark can guarantee zero contamination, these properties place ECAG-Bench at substantially lower contamination risk than benchmarks derived from competition archives or widely-used textbooks.

3 Dataset

3.1 Source Material

ECAG-Bench is derived from *Examples and Counterexamples in Elementary Algebraic Geometry*, a collection of worked examples originally written as supplementary material for learners entering algebraic geometry. The collection has been developed over several years and covers topics ranging from basic scheme theory to derived categories and moduli spaces.

The original material was written in LaTeX using a rich set of custom macros and subsequently converted to Markdown for web publication (see Section 1 for the project ecosystem). Each example was then enhanced to meet a uniform quality standard: a self-contained mathematical narrative with complete constructions, precise references, and explicit computational details. All custom macros were expanded to standard LaTeX before inclusion in the benchmark, ensuring that problems are self-contained.

3.2 Examples and Counterexamples: A Dual Structure

A distinctive feature of ECAG-Bench is its explicit dual structure of *examples* and *counterexamples*. This duality reflects a fundamental principle in mathematical pedagogy and research: understanding a concept requires seeing both what it includes and what it excludes.

Examples (243 problems, 73.2%). Example problems ask the model to construct a concrete mathematical object satisfying prescribed properties. These test *forward reasoning*: given a definition or theorem, can the model instantiate it? For instance, `ecag-0158` asks the model to construct an étale cover of the nodal cubic, requiring knowledge of étale morphisms, the structure of the nodal singularity, and how normalization provides such covers.

Counterexamples (66 problems, 19.9%). Counterexample problems ask the model to construct an object that *violates* a plausible-sounding statement. These test *boundary reasoning*: can the model identify exactly which hypothesis of a theorem is essential? For instance, one problem asks the model to exhibit a reduced scheme that is not geometrically reduced—requiring the model to understand that reducedness is not preserved under base change for non-perfect fields, and to construct a specific counterexample over $\mathbf{F}_p(T)$.

Why the 243:66 ratio? The approximately 4:1 ratio of examples to counterexamples reflects the natural structure of algebraic geometry pedagogy. In standard textbooks [Hartshorne, 1977, Vakil, 2017], the majority of illustrative material consists of positive examples that demonstrate how theories work. Counterexamples appear less frequently but are of outsized importance: they are the "stress tests" that delineate theorem boundaries. Our ratio mirrors this pedagogical reality while ensuring sufficient statistical power for comparing performance across the two types.

Illustrative contrast. Consider the concept of *flatness* in algebraic geometry. An example problem might ask:

> *Give an example of a flat family of curves over $\mathbf{A}^1$ whose general fiber is smooth but whose special fiber is singular.*

A counterexample problem on the same topic might ask:

> *Give an example of a morphism that is locally of finite type but not flat, and identify which property of the source or target is responsible.*

The first tests whether the model can construct a standard degeneration (e.g., $y^2 = x^3 - t$); the second requires understanding that flatness can fail for specific geometric reasons and producing a witness.

3.3 Dataset Statistics

Figure 1 provides a visual overview of the dataset distribution.

3.4 Topic Coverage

Beyond the chapter-level organization, ECAG-Bench covers a fine-grained set of topics within algebraic geometry. Table 3 shows the distribution across major topic areas.

The breadth of topic coverage ensures that ECAG-Bench tests algebraic geometry competence across the field rather than overweighting a single area. Problems involving *morphisms* (57) span the full range from basic properties (locally of finite type, separated) to advanced topics (étale covers, resolution of singularities). *Computations* (58) include intersection numbers, Chern classes, Hilbert polynomials, and cohomology dimensions. *Moduli* problems (34) test understanding of parameter spaces, deformation theory, and the interplay between local and global structure.

3.5 Difficulty Levels

Problems are assigned one of three difficulty levels:

Level 1 (Routine, 77 problems): The construction follows directly from definitions. A student who has read the relevant section of a standard textbook should be able to produce an answer. Example: "Give an example of a non-reduced scheme."

Level 2 (Non-trivial, 237 problems): The construction requires applying a significant theorem or combining multiple standard results. This is the level of a good homework problem in a graduate course. Example: "Construct a flat family of curves whose special fiber has a node."

Table 2: ECAG-Bench dataset statistics.

Metric	**Count**
Total entries	332
Substantive (with solution)	332
Stubs	0
By Chapter	
Basic Concepts	162
Computations	66
Moduli & Deformation	58
Cohomology	46
By Difficulty	
Level 1 (Routine)	77
Level 2 (Non-trivial theorem)	237
Level 3 (Deep/combining results)	18
By Question Type	
Example	243
Counterexample	66
Computation	14
Proof	9
By Verification Strategy	
CAS	309
Exact match	14
Rubric	9

Level 3 (Deep, 18 problems): The construction requires either deep structural insight, the combination of results from different areas, or the knowledge of a specific "trick" that is not obvious from the problem statement. Example: "Exhibit a smooth projective threefold with trivial canonical bundle that is not a product of lower-dimensional varieties."

The heavy concentration at Level 2 (71.4%) reflects our intention to test *working mathematical knowledge* rather than either elementary definitions or research-frontier results.

3.6 Content Enhancement Pipeline

The content enhancement process involved restructuring each example into a natural mathematical exposition following a consistent quality standard:

1. **Narrative form:** Each example reads as a self-contained mathematical essay rather than a template with bullet points.
2. **Complete constructions:** All computational steps are shown explicitly, allowing independent verification.
3. **Structured format:** Each entry follows a Statement → Construction/Proof → Key Insight → Prerequisites structure, facilitating both human review and automated extraction.
4. **Benchmark problem extraction:** Each example is annotated with a structured benchmark problem specification, including the problem statement, expected answer type, difficulty level, and verification strategy.

The enhancement was performed using the Anthropic API (via a custom script, `enhance_examples.py`), with each output reviewed for mathematical correctness. All 332 entries have been enhanced to

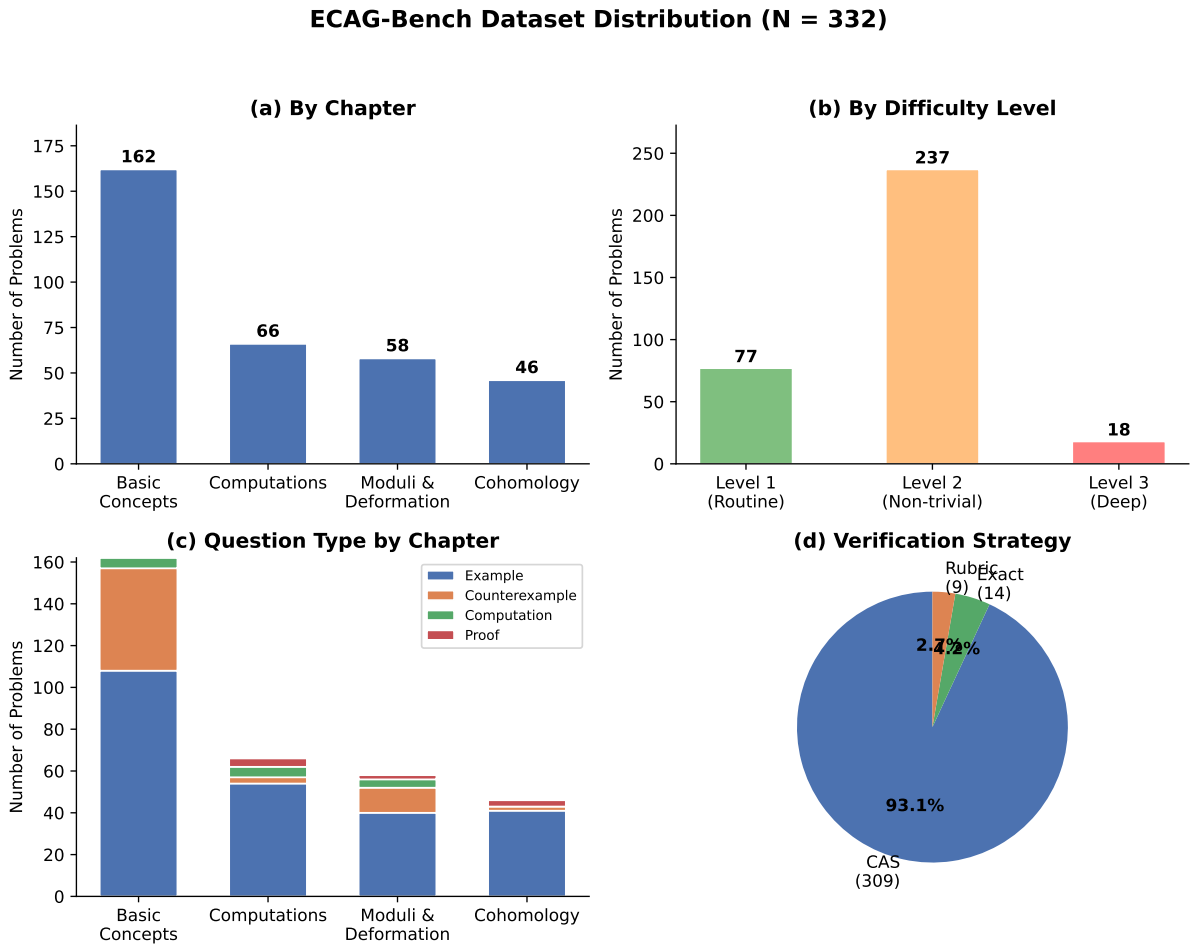

Figure 1: Distribution of ECAG-Bench problems. (a) By chapter: Basic Concepts dominates, reflecting its foundational role. (b) By difficulty: Level 2 is the majority, corresponding to graduate homework difficulty. (c) Question type breakdown by chapter, showing that counterexamples are distributed across all chapters. (d) Verification strategy: CAS verification covers 93.1% of problems.

this standard, with zero stubs remaining.

3.7 Quality Control

Each entry has been reviewed for:

- Mathematical correctness of the stated construction
- Completeness of computational details
- Accuracy of the benchmark problem formulation
- Consistency of difficulty rating across the dataset
- Validity of the assigned verification strategy
- Schema compliance (validated against a JSON Schema; see Appendix C)
- Macro expansion correctness (all custom LaTeX macros expanded to standard form)

The validation pipeline includes automated schema checking via `validate_benchmark.py`, which verifies that every entry has all required fields, difficulty levels are in $\{1, 2, 3\}$, verification strategies are valid, and cross-references to source files resolve correctly.

Table 3: Distribution of problems by topic (top 10).

Topic	**Count**
Computations (intersection theory, Chern classes, etc.)	58
Morphisms (flat, étale, proper, etc.)	57
Basic concepts (schemes, spectra, etc.)	52
Moduli spaces & deformations	34
Cohomology (sheaf, étale, de Rham)	33
Sheaves & bundles	23
Curves (genus, Jacobian, etc.)	22
Schemes (reduced, normal, regular)	20
Divisors & line bundles	18
Other (functors, groups, invariants)	15

4 Benchmark Design

4.1 Problem Types

ECAG-Bench problems fall into four categories:

Example (243): Construct a mathematical object with specified properties. ("Give an example of a flat morphism that is not smooth.")

Counterexample (66): Construct an object that violates a plausible-sounding statement. ("Give a counterexample to: every coherent sheaf on a smooth variety is locally free.")

Computation (14): Compute a specific invariant of a given object. ("Compute the Picard group of $\operatorname{Spec} \mathbb{Z}[\sqrt{-5}]$.")

Proof (9): Prove a statement about a specific object. ("Show that the Horrocks–Mumford bundle is stable.")

Figure 2 presents representative problems from each category.

4.2 Four-Layer Verification Pipeline

Since constructions in algebraic geometry are non-unique, standard answer-matching is insufficient. ECAG-Bench employs a four-layer verification pipeline, illustrated in Figure 3.

Layer 1: Parsing. The model's output is parsed to extract the claimed mathematical object. This layer uses regex and SymPy to determine whether the output is a well-formed mathematical expression (a polynomial, a variety defined by equations, a morphism given by coordinate functions, etc.). Malformed outputs receive a score of 0 immediately.

Layer 2: CAS Verification. The extracted object is verified using computer algebra systems—primarily SageMath [The Sage Developers, 2024], with Singular [Decker et al., 2024] and Macaulay2 [Grayson and Stillman, 2024] for specialized computations. The CAS checks whether the constructed object actually satisfies the required properties: genus, smoothness, dimension, cohomology vanishing, etc. This is the *core layer*: 309 of 332 problems (93.1%) use CAS verification.

Layer 3: LLM-Assisted Judgment. For problems where CAS verification is insufficient (e.g., those requiring assessment of reasoning quality or non-triviality), a rubric-based LLM judge evaluates four criteria:

- (C1) **Correctness:** Is the constructed object valid?
- (C2) **Completeness:** Are all required properties verified?
- (C3) **Non-triviality:** Is the answer non-degenerate?
- (C4) **Reasoning:** Is the justification sound?

Layer 4: Human Expert Sampling. A random subset of model responses is reviewed by a human expert in algebraic geometry to calibrate the automated layers and catch systematic errors. In our pilot evaluation (Section 5), automated LLM-based scoring was used for initial assessment, with human review of edge cases.

4.3 Verification Strategies

Each problem is assigned one of three verification strategies:

Strategy A (Exact, 14 problems, 4.2%): Problems with unique numeric or symbolic answers (e.g., "Compute $\dim H^1(X, \mathcal{O}_X)$"). Verified by exact match after normalization.

Strategy B (CAS, 309 problems, 93.1%): The model outputs a mathematical object; CAS verifies its properties deterministically. This is the primary strategy. For example, if the problem asks for a smooth curve of genus 2, the CAS checks that the given equations define a smooth curve and computes its genus.

Strategy C (Rubric, 9 problems, 2.7%): Structured scoring on correctness, completeness, non-triviality, and reasoning validity. Used for proofs and complex constructions where CAS alone is insufficient.

4.4 Counterexample Verification Challenges

Verifying counterexamples presents unique challenges compared to verifying examples. A correct counterexample must satisfy two conditions simultaneously:

1. The constructed object must be *valid*—it must be a well-defined mathematical object belonging to the appropriate category.
2. The object must *violate* the target property—the specific property that the counterexample is meant to disprove.

For instance, if the problem asks for a counterexample to "every flat morphism is smooth," the model must produce: (a) a specific morphism that is demonstrably flat (positive verification), and (b) evidence that this morphism is not smooth (negative verification). The CAS must check both conditions, which typically requires running two separate verification scripts.

This dual verification requirement makes counterexample problems inherently more complex to evaluate and may partially explain the performance gap we observe in Section 5.

4.5 Evaluation Protocol

Models are evaluated in a zero-shot, single-turn setting. Each problem is presented as a standalone prompt with the following structure:

Listing 1: Evaluation prompt template (abbreviated; full version in Appendix B).

```
You are an expert algebraic geometer.

PROBLEM: {problem_statement}

Provide a CONCRETE construction with full
justification. Your response should include:
1. The explicit mathematical object
2. Verification that it satisfies all
   required properties
3. Key insight or theorem used

Format your answer clearly with the
mathematical object stated first.
```

4.6 Scoring

Responses are scored on a three-point scale:

- **Correct (1.0):** The constructed object is valid and satisfies all stated properties. The justification is mathematically sound.
- **Partial (0.5):** The object is valid but missing some properties, or the justification is incomplete but the construction is on the right track. For counterexamples, partial credit is awarded when the object is valid but the violation is not fully demonstrated.
- **Incorrect (0.0):** The object is invalid, does not exist, or does not satisfy the required properties. Includes fabricated objects (F1), confused properties (F2), and hallucinated theorems (F5).

The scoring criteria are designed to distinguish between models that have genuine mathematical understanding (which may produce partially correct constructions) and models that produce superficially plausible but mathematically invalid outputs.

5 Experiments

We report results from a pilot evaluation designed to establish initial baselines, validate the benchmark design, and investigate the example-versus-counterexample performance gap.

5.1 Evaluation Setup

Model. We evaluate Claude Sonnet 4.6 [Anthropic, 2024], a frontier language model from Anthropic, using the Claude Code CLI (`claude -p`) in print mode with default settings. We focus on a single strong model to enable deep qualitative analysis rather than superficial comparison across many models; multi-model evaluation is planned for future work.

Subset selection. From the full 332-problem benchmark, we select a stratified random subset of 100 problems. The stratification ensures proportional representation across chapter, difficulty level, and question type, with minimum constraints of 20 counterexample problems, 20 Level 1 problems, and 10 Level 3 problems (when available) to provide statistical power for key comparisons. The resulting subset contains 70 examples, 23 counterexamples, 4 computations, and 3 proofs, distributed as: 47 Basic Concepts, 21 Computations, 17 Moduli, 15 Cohomology; and 22 Level 1, 67 Level 2, 11 Level 3.

Prompt. Each problem is presented using the prompt template described in Section 4 (full prompt in Appendix B).

Scoring. Responses were scored using a two-stage pipeline. First, an automated LLM-based scorer evaluated each response against the known solution using the 0/0.5/1 scale. The scorer was presented with the problem, known solution, and model response, and asked to classify the response as correct (1.0), partially correct (0.5), or incorrect (0.0), along with a failure mode classification (F1–F6, defined in Section 6). Problems where the model timed out (no response within 600 seconds) were automatically scored as 0.0. We discuss the limitations of this scoring approach in Section 7.

5.2 Pilot Results

Overall performance. Table 4 presents the pilot results by difficulty level.

The overall accuracy of 78% masks a steep difficulty gradient: 95% at Level 1, 81% at Level 2, and 27% at Level 3. The 68-percentage-point drop from Level 1 to Level 3 demonstrates that ECAG-Bench effectively discriminates between routine graduate-level constructions and deep

Table 4: Pilot evaluation results: Claude Sonnet 4.6 on 100-problem stratified subset. Accuracy is the proportion of scores = 1.0; Partial includes scores = 0.5. Eighteen problems timed out (counted as Incorrect).

	n	**Correct**	**Partial**	**Incorrect**	**Accuracy**
Overall	100	78	4	18	0.78
Level 1 (Routine)	22	21	0	1	0.95
Level 2 (Non-trivial)	67	54	1	12	0.81
Level 3 (Deep)	11	3	3	5	0.27

problems requiring synthesis of multiple results. Including partial credit, the model achieves an average score of 80%.

Timeout analysis. Of the 100 problems, 18 (18%) timed out without producing a response within 600 seconds. The timeout rate increases steeply with difficulty: 4.5% at Level 1, 17.9% at Level 2, and 45.5% at Level 3. Among the 82 problems that received responses, the accuracy is 95.1% (78/82), suggesting that when the model can respond, it tends to produce correct constructions—but harder problems more frequently cause the model to fail silently.

Performance by question type. Table 5 reveals a pronounced gap between examples and counterexamples.

Table 5: Pilot results by question type, showing the example–counterexample gap.

Question Type	n	**Correct**	**Partial**	**Incorrect**	**Accuracy**
Example	70	56	1	13	0.80
Counterexample	23	15	3	5	0.65
Computation	4	4	0	0	1.00
Proof	3	3	0	0	1.00

The model achieves 80% accuracy on examples but only 65% on counterexamples—a gap of 15 percentage points. This gap is consistent with our hypothesis that counterexample construction requires deeper understanding of theorem boundaries (Section 3.2). Among the 4 partial-credit responses, 3 were on counterexample problems, further suggesting that counterexamples elicit more fragile reasoning.

Performance by chapter. Table 6 shows results across the four represented chapters.

Table 6: Pilot results by chapter.

Chapter	n	**Correct**	**Accuracy**	**Partial rate**
Cohomology	15	14	0.93	0.00
Basic Concepts	47	37	0.79	0.04
Computations	21	16	0.76	0.00
Moduli	17	11	0.65	0.12

Cohomology has the highest accuracy (93%), possibly reflecting the structured and well-documented nature of cohomological computations. Moduli & Deformation has the lowest ac-

curacy (65%) and the highest partial-credit rate (12%), consistent with the more abstract and less standardized nature of the constructions required.

5.3 Case Studies

We present three case studies illustrating representative model behaviors.

Case Study 1: Successful counterexample construction (`ecag-0043`). This Level 2 problem asks for the precise difference between "locally of finite type" and "of finite type" for morphisms of schemes, along with a separating example. The model correctly identified quasi-compactness as the distinguishing property and constructed $X = \coprod_{n=0}^{\infty} \mathbb{A}_k^n \to \mathrm{Spec}(k)$ as a morphism that is locally of finite type but not of finite type. The verification correctly showed that each affine piece is a finitely generated k-algebra but the source is not quasi-compact. This demonstrates the model's strength on well-established counterexamples in the graduate curriculum.

Case Study 2: Property confusion on counterexample (`ecag-0343`, F2). This Level 3 problem concerns a $\mathbb{Z}/2\mathbb{Z}$-action on $\mathbb{P}^3$ and asks to verify geometric properties of the action and compute an invariant ring. The model substituted a different group action ($g \cdot [x : y : z : w] = [-x : -y : z : w]$) from the one specified in the problem setup, then verified properties for this substituted action. While the algebraic calculations for the substituted action were correct, the mismatch with the intended action constitutes a property confusion error: the construction is valid but answers a different question.

Case Study 3: Timeout on deep problem (`ecag-0363`). This Level 3 problem asks to compute genus-0 Gromov–Witten invariants using Schubert calculus on Grassmannians. The model timed out after 600 seconds without producing any response. This pattern—where the model fails silently on problems requiring deep synthesis of advanced techniques (Gromov–Witten theory, Schubert calculus, quantum cohomology)—accounted for 45.5% of Level 3 problems, making timeout the dominant failure mode at the highest difficulty level.

5.4 Comparison with Existing Benchmarks

Direct comparison with other benchmarks is difficult due to differences in domain and evaluation methodology. However, we note several reference points:

- FrontierMath [Glazer et al., 2024] reported that frontier models solve fewer than 2% of their research-level problems, though their problems are designed to be at the frontier of mathematical knowledge.
- On the MATH benchmark [Hendrycks et al., 2021], frontier models now achieve 90+% accuracy, indicating that competition-level mathematics is largely solved.
- On PutnamBench [Tsoukalas et al., 2024], formal theorem proving success rates remain low for the hardest problems.

ECAG-Bench occupies an intermediate position: the 78% overall accuracy and 27% Level 3 accuracy suggest that routine constructive algebraic geometry is accessible to frontier models, but problems requiring deep synthesis of multiple advanced techniques remain challenging. The steep difficulty gradient (95% → 81% → 27%) indicates that ECAG-Bench effectively spans this transition.

6 Analysis

6.1 Failure Mode Taxonomy

Based on the pilot evaluation, we define six failure modes specific to constructive algebraic geometry reasoning. While the high overall accuracy (78%) and the dominance of timeouts among failures limit our quantitative observations, the taxonomy provides a systematic framework for characterizing model weaknesses.

F1: Fabricated objects. The model produces a mathematical object that does not exist or is not well-defined. *Definition:* CAS verification reveals that the stated object does not have the claimed algebraic structure (e.g., a "smooth curve" that is singular, a "coherent sheaf" with incompatible transition functions).

F2: Property confusion. The model constructs a valid object but confuses the required properties—either verifying the wrong property, or failing to distinguish between subtly different conditions. *Pilot example (`ecag-0343`):* In a Level 3 problem about a $\mathbb{Z}/2\mathbb{Z}$-action on $\mathbb{P}^3$, the model substituted a different group action from the one specified, then correctly verified properties for this substituted action. The algebraic computations were sound, but the mismatch with the intended construction constitutes a property confusion error.

F3: Computational errors. The model follows a correct strategy but makes arithmetic or algebraic errors in computation. This class of errors is particularly insidious because the reasoning structure is correct, and only explicit CAS computation reveals the mistake.

F4: Trivial or degenerate answers. The model produces a technically correct but trivially degenerate answer that demonstrates no mathematical understanding. *Definition:* Answers that use identity morphisms, zero sheaves, or other universal constructions that trivially satisfy conditions without illustrating the mathematical concept.

F5: Hallucinated theorems. The model invokes a theorem that does not exist or misattributes a result. This failure mode is expected to be concentrated at higher difficulty levels, where the model is more likely to encounter knowledge gaps that it fills with plausible-sounding fabrications.

F6: Incomplete verification. The model constructs a plausible object and provides some justification but fails to verify all required properties. *Pilot example (`ecag-0095`):* On a Level 3 problem about blow-ups, the model provided a textbook-perfect construction of the blow-up of $\mathbb{A}^2$ at the origin, but the problem asked for a more specific construction that the model did not fully address, resulting in partial credit.

Observed failure mode distribution. Among the 82 problems that received responses, only 4 were scored below 1.0. Of these, 3 received F6 (incomplete verification) and 1 received F2 (property confusion). The remaining 18 failures were timeouts, for which no failure mode can be assigned since the model produced no response. This sparsity limits failure mode analysis and suggests that future evaluations with larger samples and multiple models will be needed to fully populate the taxonomy.

6.2 Difficulty Calibration

The pilot results show a clear monotonic relationship between assigned difficulty and model accuracy:

	Level 1	Level 2	Level 3
n	22	67	11
Accuracy	0.95	0.81	0.27
Avg. score	0.95	0.81	0.41
Timeout rate	4.5%	17.9%	45.5%

This validates the difficulty calibration: problems rated harder by a human expert are indeed substantially harder for the model. The steepest drop is from Level 2 to Level 3 (54 percentage points), corresponding to the transition from problems requiring a single non-trivial theorem to those requiring deep synthesis of multiple advanced results.

The timeout rate provides an additional lens: at Level 3, nearly half the problems cause the model to fail without producing any response, compared to fewer than 5% at Level 1. This suggests that model confidence (or generation feasibility) correlates strongly with problem difficulty.

We note that difficulty calibration could also be influenced by the frequency of similar problems in training data. Level 1 problems (e.g., "give an example of a non-reduced scheme") may appear frequently in online educational materials, while Level 3 problems are closer to research-level constructions with fewer training exemplars.

6.3 Chapter-Level Analysis

Performance varies across chapters:

Chapter	n	**Accuracy**	**Partial rate**	**Timeout rate**
Cohomology	15	0.93	0%	6.7%
Basic Concepts	47	0.79	4.3%	17.0%
Computations	21	0.76	0%	23.8%
Moduli	17	0.65	11.8%	23.5%

Several patterns emerge:

1. **Cohomology** has the highest accuracy (93%), possibly reflecting the structured, algorithm-like nature of cohomological computations (spectral sequences, long exact sequences) that aligns well with sequential reasoning.
2. **Basic Concepts** has moderate accuracy (79%) despite being the most elementary topic, partly because this chapter contains many Level 2 problems requiring non-trivial constructions beyond textbook definitions.
3. **Computations** has a 24% timeout rate, the highest among all chapters, reflecting the difficulty of multi-step algebraic calculations (Chern classes, intersection products, Hodge numbers).
4. **Moduli & Deformation** has the lowest accuracy (65%) and highest partial-credit rate (12%), suggesting that the model struggles with the abstract constructions required in deformation theory (e.g., computing tangent spaces to moduli functors, constructing versal deformations).

6.4 Example vs. Counterexample Performance Gap

The 15-percentage-point gap between example accuracy (80%) and counterexample accuracy (65%) is a key finding of the pilot evaluation.

Structural differences. Among partial-credit responses, 3 of 4 were on counterexample problems, suggesting that counterexample construction more often leads to partially correct responses where the right idea is present but execution is flawed. The failure mode on these partial counterexample responses was predominantly F6 (incomplete verification) and F2 (property confusion): the model identified the relevant mathematical territory but could not precisely delineate the boundary between a theorem's hypotheses and its conclusion.

Timeout distribution. Among the 18 timeouts, 13 were on example problems and 5 on counterexample problems. As a fraction of each type: $13/70 = 18.6\%$ of examples timed out vs. $5/23 = 21.7\%$ of counterexamples. The slightly higher counterexample timeout rate, combined with the lower accuracy among completed responses, suggests that counterexamples are harder for the model both in terms of producing a response and in terms of producing a correct one.

Qualitative observation. In successful counterexample constructions, the model typically follows a clear pattern: (1) state the target property, (2) identify the critical hypothesis, (3) construct an object where that specific hypothesis fails, and (4) verify both that the object is valid and that the property indeed fails. In unsuccessful attempts, the model often conflates the hypothesis and conclusion, or constructs an object that is loosely related to the topic without precisely targeting the boundary.

This pattern is consistent with the Lakatosian view [Lakatos, 1976] that counterexample construction requires "concept stretching"—pushing a definition to its limits to find where it breaks.

6.5 Limitations of Self-Evaluation

A significant limitation of this pilot is that both the problem solver and the scorer are LLMs from the same model family (Claude Sonnet 4.6). Among the 82 problems that received responses, 78 (95.1%) were scored as correct. This high concordance rate could reflect either genuine high accuracy or systematic bias in self-evaluation.

Several factors suggest caution:

- The scorer may share the solver's blindspots, consistently rating as correct a construction that both models find plausible but that a human expert would flag as flawed.
- The scorer was given the known solution, which may bias it toward confirming any response that follows a similar structure.
- With only 4 non-timeout failures among 82 responses, the failure mode taxonomy remains sparsely populated.

Future work will address this through human expert scoring of a stratified sample, cross-model evaluation (using different models as solver and scorer), and CAS-based automated verification where possible.

7 Conclusion

We have introduced ECAG-Bench, a benchmark of 332 constructive problems in algebraic geometry designed to evaluate a mode of mathematical reasoning—object construction—that is central to mathematical practice but underrepresented in existing LLM benchmarks.

Key findings. Our pilot evaluation of Claude Sonnet 4.6 on a stratified 100-problem subset reveals:

1. **A steep difficulty gradient:** The model achieves 78% accuracy overall, but this masks a 68-percentage-point drop from Level 1 (95%) to Level 3 (27%), confirming that ECAG-

Bench effectively discriminates between routine constructions and deep problems requiring synthesis of advanced techniques.

2. **Counterexamples are harder than examples:** The 15-percentage-point gap (80% vs. 65%) confirms that understanding theorem boundaries requires deeper reasoning than applying theorems directly.
3. **Timeouts as a failure mode:** 18% of problems caused the model to time out (>600s), with the timeout rate reaching 45.5% at Level 3, suggesting that the hardest problems cause the model to fail silently before producing any response.
4. **Chapter-dependent performance:** Accuracy ranges from 93% (Cohomology) to 65% (Moduli), reflecting the varying difficulty of constructive reasoning across algebraic geometry subfields.
5. **Failure modes are structured:** The six failure modes (F1–F6) provide a systematic taxonomy for characterizing model weaknesses, with property confusion (F2) and incomplete verification (F6) observed in the pilot.

Limitations. Our evaluation has several limitations that should be addressed in future work:

- **Single model:** We evaluate only Claude Sonnet 4.6. Multi-model comparison (GPT-4o, Gemini 2.5 Pro, DeepSeek-R1 [DeepSeek-AI, 2025], Llemma [Azerbayev et al., 2024]) is needed to determine whether the observed patterns are model-specific.
- **Self-evaluation bias:** Both the problem solver and the automated scorer are LLMs from the same model family, introducing potential concordance bias. Among the 82 problems that received responses, 95.1% were scored as correct, which may reflect systematic overrating. Human expert scoring is needed to validate these results (see Section 6.5).
- **Single-turn only:** Multi-turn interaction, where the model can refine constructions based on CAS feedback, may significantly improve performance and is an important evaluation modality to explore.
- **Domain specificity:** Results in algebraic geometry may not generalize to other mathematical domains, though we expect similar example/counterexample patterns.
- **CAS coverage:** While 93.1% of problems use CAS verification, the CAS cannot verify all properties, and some verifications require specialized scripts that have not yet been fully implemented.

Future work. We plan several extensions:

1. **Full-scale evaluation with human scoring:** Evaluate multiple frontier models on the complete 332-problem benchmark with human expert scoring to establish unbiased baselines.
2. **Multi-turn interaction:** Evaluate models in an interactive setting where they receive CAS feedback and can revise their constructions.
3. **Lean 4 integration:** Formalize a subset of problems in Lean 4, enabling formal verification as an additional evaluation layer.
4. **Fine-tuning studies:** Investigate whether fine-tuning on algebraic geometry textbooks [Hartshorne, 1977, Vakil, 2017, Eisenbud, 1995] or the accompanying ECAG-Bench textbook improves constructive reasoning.
5. **Community contributions:** The open-source nature of the project (CC-BY-4.0 license) invites community contributions of new problems, verification scripts, and evaluation results.
6. **Cross-domain extension:** Develop analogous benchmarks for arithmetic geometry, algebraic topology, and commutative algebra to investigate whether the example/counterexample performance gap generalizes.

Broader impact. Constructive mathematical reasoning is a prerequisite for using LLMs as research tools in mathematics. If models cannot construct examples and counterexamples, they cannot assist mathematicians in the exploratory phase of research—the phase where intuition is built and conjectures are tested. The pronounced difficulty of counterexample construction is particularly significant: as Lakatos [Lakatos, 1976] argued, mathematical progress depends on the ability to find refutations, and this ability appears to be a current blind spot for language models. ECAG-Bench provides a concrete measure of progress toward closing this gap.

References

Anthropic. The Claude model family. 2024. Model card available at `https://www.anthropic.com`.

Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q Jiang, Jia Deng, and Stella Biderman. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023.

Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q Jiang, Jia Deng, Stella Biderman, and Sean Welleck. Llemma: An open language model for mathematics. *ICLR*, 2024.

Mislav Balunović, Nikola Jovanović, Robin Staab, Mark Vero, and Martin Vechev. MathArena: Evaluating llms on uncontaminated math competitions. 2025. ETH Zurich, `https://matharena.ai/`.

Katherine M Collins, Albert Q Jiang, Simon Frieder, Lionel Wong, Miri Zilka, Umang Bhatt, Thomas Lukasiewicz, Yuhuai Wu, Joshua B Tenenbaum, William Hart, et al. Evaluating language models for mathematics through interactions. *arXiv preprint arXiv:2306.01694*, 2024.

Wolfram Decker, Gert-Martin Greuel, Gerhard Pfister, and Hans Schönemann. SINGULAR 4-4-0 — a computer algebra system for polynomial computations. 2024. `https://www.singular.uni-kl.de`.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

David Eisenbud. *Commutative Algebra with a View Toward Algebraic Geometry*. Springer, 1995.

Simon Frieder, Luca Pinchetti, Ryan-Rhys Griffiths, Tommaso Salvatori, Thomas Lukasiewicz, Philipp Petersen, Alexis Chevalier, and Julius Berner. Mathematical capabilities of ChatGPT. *NeurIPS*, 2023.

Elliot Glazer, Ege Erdil, Tamay Besiroglu, Diego Chicharro, Evan Chen, Alex Gunning, Caroline Falkman Olsson, Jean-Stanislas Denain, Anson Ho, Emily de Oliveira Santos, et al. Frontiermath: A benchmark for evaluating advanced mathematical reasoning in ai. *arXiv preprint arXiv:2411.04872*, 2024.

Daniel R. Grayson and Michael E. Stillman. Macaulay2, a software system for research in algebraic geometry. 2024. Available at `https://macaulay2.com`.

Phillip Griffiths and Joseph Harris. *Principles of Algebraic Geometry*. John Wiley & Sons, 1978.

Robin Hartshorne. *Algebraic Geometry*. Springer, 1977.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *NeurIPS*, 2021.

Heisuke Hironaka. Resolution of singularities of an algebraic variety over a field of characteristic zero. *Annals of Mathematics*, 79(1–2):109–326, 1964.

Geoffrey Horrocks and David Mumford. A rank 2 vector bundle on $\mathbf{P}^4$ with 15,000 symmetries. *Topology*, 12(1):63–81, 1973.

Imre Lakatos. *Proofs and Refutations: The Logic of Mathematical Discovery.* Cambridge University Press, 1976.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *NeurIPS*, 2022.

Yiming Liu et al. AMBER: Construction and verification benchmark for mathematical reasoning in Lean 4. *arXiv preprint arXiv:2602.01291*, 2025.

Masayoshi Nagata. On the 14th problem of Hilbert. *American Journal of Mathematics*, 81(3): 766–772, 1959.

The Stacks project authors. The stacks project. `https://stacks.math.columbia.edu`, 2024.

The Sage Developers. *SageMath, the Sage Mathematics Software System (Version 10.2)*, 2024. `https://www.sagemath.org`.

Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving Olympiad geometry without human demonstrations. *Nature*, 625:476–482, 2024.

George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amitayush Thakur, and Swarat Chaudhuri. PutnamBench: Evaluating neural theorem-provers on the putnam mathematical competition. *arXiv preprint arXiv:2407.11214*, 2024.

Ravi Vakil. *The Rising Sea: Foundations of Algebraic Geometry.* 2017. Available at `https://math.stanford.edu/~vakil/216blog/`.

Sean Welleck, Jiacheng Liu, Ximing Lu, Hannaneh Hajishirzi, and Yejin Choi. NaturalProver: Grounded mathematical proof generation with language models. *NeurIPS*, 2022.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: A cross-system benchmark for formal Olympiad-level mathematics. *ICLR*, 2022.

Yifan Zheng et al. GHOSTS: Graduate-level holistic open-ended science tasks. In *NeurIPS 2023 Datasets and Benchmarks Track*, 2023.

Wanjun Zhong, Ruixiang Cui, Yiduo Guo, Yaobo Liang, Shuai Lu, Yanlin Wang, Amin Saied, Weizhu Chen, and Nan Duan. AGIEval: A human-centric benchmark for evaluating foundation models. *arXiv preprint arXiv:2304.06364*, 2023.

A Extended Problem Examples

We present six complete problem–solution pairs from ECAG-Bench, selected to illustrate the range of problem types, difficulty levels, and mathematical content.

A.1: Counterexample — Infinite Disjoint Union (Level 2)

ecag-0024 — Basic Concepts

Problem: Let k be a field. Is the scheme $\coprod_{i=1}^{\infty} \operatorname{Spec}(k)$ affine? Justify your answer by identifying which property of affine schemes it fails to satisfy.

Solution (abbreviated): The scheme $X = \coprod_{i=1}^{\infty} \operatorname{Spec}(k)$ is not affine. The key property that fails is *quasi-compactness*: affine schemes are quasi-compact, but X has the discrete topology on an infinite set, so the cover $\{U_i = \operatorname{Spec}(k)\}$ has no finite subcover.
Alternatively, if X were affine, we would have $X \cong \operatorname{Spec}(R)$ for $R = \Gamma(X, \mathcal{O}_X) \cong \prod_{i=1}^{\infty} k$. But $\operatorname{Spec}(\prod_{i=1}^{\infty} k)$ is a connected scheme (it has no non-trivial idempotents besides those coming from finite products), contradicting the fact that X is disconnected with infinitely many connected components.

Type: Counterexample **Difficulty:** 2 **Verification:** CAS (check quasi-compactness of global sections ring)

A.2: Example — Hopf Algebra Structures in Char p (Level 1)

ecag-0003 — Basic Concepts

Problem: Let k be a field of characteristic $p > 0$. Write down explicitly the Hopf algebra structures (comultiplication, counit, antipode) on $k[x]/(x^p - 1)$ for μ_p and on $k[x]/(x^p)$ for α_p. Using the substitution $u = x - 1$ to identify the underlying algebras as $k[u]/(u^p)$, show that $\mu_p \ncong \alpha_p$ as group schemes.

Solution (abbreviated): Under $u = x - 1$, both algebras become $k[u]/(u^p)$, but the comultiplications differ:

$$\Delta_\mu(u) = u \otimes 1 + 1 \otimes u + u \otimes u \quad (\text{from } \mu_p)$$
$$\Delta_\alpha(u) = u \otimes 1 + 1 \otimes u \quad (\text{from } \alpha_p)$$

Any k-algebra automorphism of $k[u]/(u^p)$ has the form $u \mapsto \lambda u +$ (higher terms) with $\lambda \neq 0$, but Δ_μ has a quadratic term $u \otimes u$ that cannot be eliminated by any such automorphism.

Type: Example **Difficulty:** 1 **Verification:** CAS (verify Hopf algebra axioms)

A.3: Counterexample — Locally Finite Type vs. Finite Type (Level 2)

ecag-0043 — Basic Concepts

Problem: State the precise difference between "locally of finite type" and "of finite type" for a morphism of schemes. Give an example of a morphism that is locally of finite type but not of finite type, and identify which condition fails.

Solution (abbreviated): A morphism $f : X \to Y$ is *locally of finite type* if for every $x \in X$, there exist affine neighborhoods $U = \operatorname{Spec}(A)$ of $f(x)$ and $V = \operatorname{Spec}(B) \subseteq f^{-1}(U)$ of x such that B is a finitely generated A-algebra. It is *of finite type* if additionally f is quasi-compact.
The morphism $\coprod_{i \in I} \mathbf{A}^1_k \to \mathbf{A}^1_k$ (identity on each component, I infinite) is locally of finite type but not of finite type, because the source is not quasi-compact.

Type: Counterexample **Difficulty:** 2 **Verification:** CAS

A.4: Example — Étale Cover of Nodal Cubic (Level 2)

ecag-0158 — Basic Concepts

Problem: Let k be an algebraically closed field with $\mathrm{char}(k) \neq 2$, and let $Y = \mathrm{Spec}(k[x,y]/(y^2 - x^2(x+1)))$ be the nodal cubic. Construct a degree-2 étale morphism $f : X \to Y$ and compute the fibers over the node and a smooth point.

Solution (abbreviated): Define $X = \mathrm{Spec}(k[s,t]/(t^2 - (s^2-1)^2))$ with f given by $x \mapsto s^2-1$, $y \mapsto st$. The fiber over the node $(0,0)$ is $\mathrm{Spec}(k[s,t]/(s^2-1, t^2-(s^2-1)^2, st)) = \{(1,0),(-1,0)\}$, a reduced scheme of two points. At smooth points, f is étale of degree 2 (one checks that $\Omega_{X/Y} = 0$ away from the node).

Type: Example **Difficulty:** 2 **Verification:** CAS (verify étale condition, compute fibers)

A.5: Computation — Horrocks–Mumford Bundle (Level 2)

ecag-0230 — Computations

Problem: Let F be the Horrocks–Mumford bundle on $\mathbf{P}^4$ with $c_1(F) = 5h$ and $c_2(F) = 10h^2$. Using HRR, compute $\chi(F(n-5))$ as a closed-form polynomial in n.

Solution (abbreviated): The Chern character of $F(m)$ (where $m = n-5$) is computed from c_1 and c_2 using the splitting principle, then paired with the Todd class of $\mathbf{P}^4$. The result is a degree-4 polynomial in n, which at $n = 5$ yields $\chi(F) = h^0 - h^1 = 4 - 2 = 2$, consistent with known cohomology.

Type: Computation **Difficulty:** 2 **Verification:** Exact match + CAS

A.6: Example — Proper Birational to Normal (Level 2)

ecag-0019 — Basic Concepts

Problem: Let $f : X \to Y$ be a proper birational morphism with Y normal. Prove that $f_*\mathcal{O}_X \cong \mathcal{O}_Y$. Which property of normal schemes is used?

Solution (abbreviated): The structure map $\mathcal{O}_Y \to f_*\mathcal{O}_X$ is injective (since f is birational and dominant). For surjectivity, any section $s \in (f_*\mathcal{O}_X)(U) = \mathcal{O}_X(f^{-1}(U))$ restricts to a rational function on U that is regular on the dense open $f^{-1}(U \cap V)$ where f is an isomorphism. Since Y is normal, the algebraic Hartogs lemma (Serre's $R_1 + S_2$ criterion) extends s to a regular function on U.

Type: Example **Difficulty:** 2 **Verification:** Rubric

B Evaluation Prompt and Scoring Rubric

B.1: Full Evaluation Prompt

```
You are an expert algebraic geometer with deep
knowledge of scheme theory, sheaf cohomology,
```

```
moduli spaces, and computational algebraic geometry.

PROBLEM:
{problem}

INSTRUCTIONS:
Provide a CONCRETE construction with full
mathematical justification. Your response should
include:

1. **The explicit mathematical object**: State the
   object clearly (e.g., a specific polynomial,
   ring, scheme, morphism, sheaf, or cohomology
   class).

2. **Verification of all required properties**: For
   each property mentioned in the problem, provide
   a rigorous argument that your object satisfies
   (or, for counterexamples, violates) it.

3. **Key insight or theorem used**: Identify the main
   mathematical idea underlying your construction.

FORMAT:
- State your answer at the very beginning, clearly
  labeled.
- Then provide the detailed justification.
- Use standard LaTeX notation for mathematical
  expressions.
- If the problem asks for a counterexample,
  explicitly state which hypothesis fails and why.
```

B.2: Scoring Rubric

Score 1.0 (Correct):

- (C1) The constructed object is well-defined and belongs to the correct mathematical category.
- (C2) All properties stated in the problem are rigorously verified (or, for counterexamples, the target property is rigorously shown to fail).
- (C3) The answer is non-trivial and non-degenerate.
- (C4) The mathematical reasoning is sound and references to standard results are accurate.

Score 0.5 (Partial):

- The object is valid (C1 satisfied) but one or more of C2–C4 is partially satisfied.
- Typical cases: correct object with incomplete verification, or correct approach with a computational error that does not invalidate the construction.

Score 0.0 (Incorrect):

- The object is invalid (C1 fails), or the properties are not satisfied (C2 fails fundamentally).
- Includes fabricated objects (F1), reversed properties (F2), and reliance on non-existent theorems (F5).

C Dataset Schema and Access

C.1: JSON Schema

Each entry in the ECAG-Bench dataset (`benchmark/data/ecag_bench.jsonl`) conforms to the following schema:

```
{
  "id": "ecag-XXXX",
  "problem": "Problem statement in LaTeX",
  "solution": "Full solution text",
  "answer": "Closed-form answer or null",
  "answer_type": "example|counterexample|
                 closed_form|proof|true_false",
  "difficulty": 1|2|3,
  "chapter": "basic_concepts|cohomology|
             computations|moduli|relative",
  "topic": "...",
  "subtopic": "...",
  "question_type": "example|counterexample|
                   computation|proof|true_false",
  "tags": ["..."],
  "source": "Original|Hartshorne X.Y.Z|...",
  "prerequisites": ["..."],
  "msc_codes": ["14XXX"],
  "verification_strategy": "exact|cas|rubric|hybrid",
  "verification_script": "path or null",
  "accepts_multiple": true|false,
  "is_stub": false,
  "latex_source": "path to .tex file"
}
```

Difficulty levels: 1 = routine (follows from definitions), 2 = non-trivial (requires significant theorem), 3 = deep (combines multiple areas or requires specialized knowledge).

C.2: Access

- **GitHub:** `https://github.com/Waerden001/Examples_and_Counterexamples_in_Elementary_Algebraic_Geometry`
- **Website:** The accompanying textbook is available as a searchable website via GitHub Pages.
- **License:** CC-BY-4.0

D Case Study Full Outputs

This appendix provides representative model outputs from the pilot evaluation discussed in Section 5. All outputs are from Claude Sonnet 4.6 via `claude -p`.

D.1: Successful Counterexample (`ecag-0043`)

Problem: State the precise difference between "locally of finite type" and "of finite type" for a morphism of schemes. Give an example of a morphism that is locally of finite type but not of finite type, and identify which condition fails.

Type: Counterexample **Difficulty:** Level 2 **Score:** 1.0

Model output (abbreviated):

Answer: The precise difference lies in **quasi-compactness**: finite type = locally of finite type + quasi-compact.

Counterexample: Let $X = \coprod_{n=0}^{\infty} \mathbb{A}_k^n$, $Y = \mathrm{Spec}(k)$, $f : X \to Y$ the structure morphism. Then f is locally of finite type but not of finite type; the failing condition is quasi-compactness.

Verification — locally of finite type: Take the affine open cover $\{V = \mathrm{Spec}(k)\}$ of Y. Then $f^{-1}(V) = X = \coprod_{n\geq 0} \mathrm{Spec}(k[t_1, \ldots, t_n])$. Each $k[t_1, \ldots, t_n]$ is a finitely generated k-algebra, so f is locally of finite type.

Verification — not quasi-compact: The open cover $\{U_n\}_{n\geq 0}$ where $U_n = \mathrm{Spec}(k[t_1, \ldots, t_n])$ has no finite subcover, since each U_n is a distinct connected component.

Key insight: The distinction between "locally" and "globally" properties of morphisms typically involves a quasi-compactness condition.

The model correctly identified quasi-compactness as the distinguishing property, constructed a standard separating example, and verified both the positive and negative conditions rigorously.

D.2: Property Confusion (`ecag-0343`, F2)

Problem: In the setup above, verify that $g \cdot P = Q$, $g \cdot C = D$, and that the fixed locus of the G-action on $\mathbb{P}^3$ is exactly $l_1 \cup l_2$. Compute the invariant ring $k[x, y, z, w]^G$ and verify the relation $v_0 v_1 = v_2^2$.

Type: Counterexample **Difficulty:** Level 3 **Score:** 0.5 **Failure mode:** F2

Model output (abbreviated):

Setup: Let $G = \mathbb{Z}/2\mathbb{Z} = \{e, g\}$, with g acting on $\mathbb{P}_k^3$ by: $g \cdot [x : y : z : w] := [-x : -y : z : w]$ corresponding to $\sigma = \mathrm{diag}(-1, -1, 1, 1)$.

Verification $g \cdot P = Q$: $g \cdot P = g \cdot [1 : 0 : -1 : 0] = [-1 : 0 : -1 : 0] = [1 : 0 : 1 : 0] = Q$. ✓

Invariant ring: The degree-d invariants of $k[x, y, z, w]$ under this action consist of monomials $x^a y^b z^c w^d$ with $a + b$ even. Setting $v_0 = x^2, v_1 = y^2, v_2 = xy, v_3 = z, v_4 = w$, the invariant ring is $k[v_0, v_1, v_2, v_3, v_4]/(v_0 v_1 - v_2^2)$. ✓

Analysis: The model substituted a different group action ($g \cdot [x : y : z : w] = [-x : -y : z : w]$) from the one specified in the problem setup (which is $g \cdot [x : y : z : w] = [y : x : w : z]$, a transposition). While the algebraic computations for the substituted action were internally consistent, the mismatch with the intended construction means the results do not answer the stated question. This exemplifies F2 (property confusion): the model demonstrates algebraic competence but applies it to the wrong mathematical object.

D.3: Timeout on Deep Problem (`ecag-0363`)

Problem: Using genus-0 Gromov–Witten invariants and Schubert calculus on $\mathrm{Gr}(2, n + 1)$, compute $\langle H^i | H^j \rangle_{0,d}$ and verify the quantum product formula.

Type: Example **Difficulty:** Level 3 **Score:** 0.0 (timeout)

The model timed out after 600 seconds without producing any response. This problem requires synthesizing Gromov–Witten theory, Schubert calculus, and quantum cohomology—a combination of techniques at the frontier of the represented material. The timeout pattern was common at Level 3, where 45.5% of problems caused the model to fail silently, suggesting that the model's inability to produce even a partial response on such problems reflects a fundamental gap in its ability to combine deep techniques from multiple areas of algebraic geometry.

Example (`ecag-0158`, Level 2, Basic Concepts)

Problem: Let k be an algebraically closed field with $\mathrm{char}(k) \neq 2$, and let $Y = \mathrm{Spec}(k[x,y]/(y^2 - x^2(x+1)))$ be the nodal cubic. Construct an étale cover $f : X \to Y$ by exhibiting an explicit morphism, and compute the degree of f at the node and at a smooth point.

Verification: CAS checks that f is étale at smooth points and computes fiber cardinalities. Key properties: f is finite, locally of finite presentation, and unramified away from the node.

Counterexample (`ecag-0005`, Level 2, Basic Concepts)

Problem: Let $k = \mathbf{F}_p(T)$ and $K = \mathbf{F}_p(T^{1/p})$. Explicitly exhibit a nonzero nilpotent element in $K \otimes_k K$ and prove that its p-th power is zero. What is the nilpotency index of this element?

Verification: CAS verifies $\alpha = 1 \otimes T^{1/p} - T^{1/p} \otimes 1 \neq 0$ and $\alpha^p = 0$ in $K[X]/(X - T^{1/p})^p$. This is a counterexample to "reduced $\Rightarrow$ geometrically reduced" over non-perfect fields.

Computation (`ecag-0230`, Level 2, Computations)

Problem: Let F be the Horrocks–Mumford bundle on $\mathbf{P}^4$ with $c_1(F) = 5h$ and $c_2(F) = 10h^2$. Using the Hirzebruch–Riemann–Roch theorem, compute $\chi(F(n-5))$ as a closed-form rational function of n.

Verification: Exact match against the closed-form expression. CAS independently computes the Euler characteristic via the Chern character and Todd class.

Proof (`ecag-0240`, Level 2, Computations)

Problem: On the Hirzebruch surface Σ_2, determine all pairs $(p,q) \in \mathbf{Z}^2$ with $-3 \leq p \leq 3$ and $-5 \leq q \leq 5$ for which $H^1(\mathcal{O}_{\Sigma_2}(p,q)) \neq 0$.

Verification: Rubric-based scoring on (C1) correct identification of non-vanishing pairs, (C2) complete enumeration, (C3) valid cohomological argument, (C4) use of appropriate vanishing theorem.

Figure 2: Representative problems from each category of ECAG-Bench. Each problem requires constructing or computing a specific mathematical object, with verification tailored to the problem type.

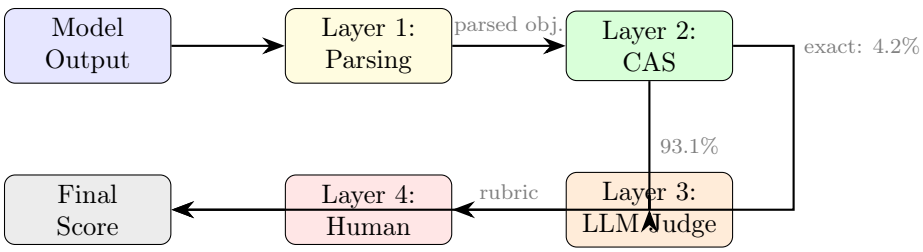

Figure 3: The four-layer verification pipeline. Most problems (93.1%) are verified by CAS at Layer 2; rubric-based problems pass through LLM judgment (Layer 3) and sampled human review (Layer 4).